

07 · Base de datos

Documentacion del sistema

Version analizada: **0.1.0**

Commit analizado: **5e91229**

Fecha de generacion: **2026-08-27**

Generado a partir de 07-database.md — los Markdown son la fuente unica.
No editar este PDF: editar el Markdown y regenerar.

Contenido de este documento

1. Resumen
2. Conexión
3. Esquema
4. Diagrama entidad-relación
5. Relación tabla ↔ código
6. Consultas relevantes
7. Transacciones
8. Retención y crecimiento
9. Datos sensibles almacenados
10. Respaldo y recuperación
11. Bases de datos ajenas que el producto lee
12. Verificación manual del contenido

Todo lo que RootCause persiste: dónde, con qué esquema, con qué consultas y con qué política de retención. Incluye también las dos bases **ajenas** que el producto lee sin escribir nunca: las **TCC.db** de macOS.

1. Resumen

Aspecto	Valor
Motor	SQLite 3, embebido en el binario (rusqlite con bundled)
Archivo	<code>~/Library/Application Support/RootCauseInspector/rootcause-history.db</code>
Tablas	5
Índices explícitos	1
Claves foráneas	Ninguna
Migraciones	No hay: CREATE TABLE IF NOT EXISTS en cada arranque
ORM	No; SQL literal en <code>src/services/persistence.rs</code>
Acceso concurrente	Una conexión por operación; sin pool
Cifrado en reposo	No propio; depende de FileVault
Salida a red	Ninguna: la base nunca se envía a ningún sitio

2. Conexión

`PersistenceStore::new(app_name):`

1. Resuelve `dirs::data_local_dir()` con respaldo `dirs::data_dir()`. En macOS, ambas apuntan a `~/Library/Application Support`.
2. Le añade `app_name` (`meta::APP_DIR = "RootCauseInspector"`) y crea el directorio con `create_dir_all`.
3. Compone `rootcause-history.db` y llama a `ensure_schema()`.

No hay cadena de conexión, ni usuario, ni contraseña, ni configuración de red: **es un archivo local del usuario**. Si el directorio no se puede crear, `InspectorService::new` falla y quien llama lo explica.

3. Esquema

3.1 snapshots — historial de capturas

Resumen compacto de cada captura, para tendencias y para la sección Historial.

Columna	Tipo	Nulo	Por defecto	Descripción
<code>id</code>	INTEGER	No	autoincremental	Clave primaria
<code>collected_at</code>	TEXT	No	—	Instante de la captura, RFC 3339 en UTC

<code>cpu_usage</code>	REAL	No	—	CPU global en porcentaje
<code>memory_used_gb</code>	REAL	No	—	Memoria usada en GB
<code>memory_total_gb</code>	REAL	No	—	Memoria total en GB
<code>cache_total_mb</code>	REAL	No	—	Total medido de cachés en MB
<code>network_rx_mb_delta</code>	REAL	No	—	Recepción de red del intervalo en MB
<code>network_tx_mb_delta</code>	REAL	No	—	Transmisión de red del intervalo en MB
<code>io_read_mb_delta</code>	REAL	No	—	Lectura de disco del intervalo en MB
<code>io_write_mb_delta</code>	REAL	No	—	Escritura de disco del intervalo en MB
<code>dominant_process</code>	TEXT	No	—	<code>nombre (pid)</code> del primer proceso ordenado, o <code>"Sin datos"</code>
<code>alerts_json</code>	TEXT	No	—	Array JSON con las alertas de esa captura
<code>created_at</code>	TEXT	No	<code>CURRENT_TIMESTAMP</code>	Marca de inserción de SQLite

Notas. `alerts_json` guarda el array completo de `Alert` serializado. `load_recent` lo deserializa a `Vec<serde_json::Value>` solo para contar (`alerts_count`) y buscar si alguna tiene `severity == "Critical"` (`has_critical`); no reconstruye los objetos.

3.2 incidents — incidentes correlacionados

Columna	Tipo	Nulo	Descripción
<code>id</code>	INTEGER	No	Clave primaria autoincremental
<code>incident_id</code>	TEXT	No	<code>inc-<epoch_ms></code> , identidad temporal del incidente
<code>fingerprint</code>	TEXT	No	<code>kind\ proceso\ título</code> , clave de deduplicación
<code>collected_at</code>	TEXT	No	RFC 3339 en UTC
<code>severity</code>	TEXT	No	<code>Healthy</code> , <code>Warning</code> o <code>Critical</code> (<code>format!("{}",{:?})</code>)
<code>kind</code>	TEXT	No	Tipo del incidente, heredado de la anomalía dominante
<code>title</code>	TEXT	No	Título legible
<code>summary</code>	TEXT	No	Resumen de una o dos frases
<code>payload_json</code>	TEXT	No	<code>IncidentSummary</code> completo serializado

<code>created_at</code>	TEXT	No	<code>CURRENT_TIMESTAMP</code>
-------------------------	------	----	--------------------------------

Notas. Las columnas escalares existen para consultar sin deserializar; la verdad completa está en `payload_json`, que es lo que leen `load_recent_incidents` y `latest_incident`. El enriquecimiento de IA se guarda actualizando ese JSON (`update_incident_ai`), no en columnas nuevas.

Índice: `idx_incidents_incident_id` sobre `incidents(incident_id)`, que sirve a `load_incident_by_id`.

3.3 `audit_log` — auditoría de acciones

Columna	Tipo	Nulo	Descripción
<code>id</code>	INTEGER	No	Clave primaria autoincremental
<code>occurred_at</code>	TEXT	No	RFC 3339 en UTC
<code>action</code>	TEXT	No	Identificador de la acción
<code>target</code>	TEXT	No	Objeto de la acción: PID, ruta, IP, recuento
<code>success</code>	INTEGER	No	<code>1</code> correcto, <code>0</code> fallido
<code>detail</code>	TEXT	No	Mensaje de éxito o texto del error
<code>created_at</code>	TEXT	No	<code>CURRENT_TIMESTAMP</code>

Acciones registradas (valores reales del código):

<code>action</code>	Origen	Cuándo
<code>accept-persistence-baseline</code>	<code>inspector.rs</code>	Se acepta la persistencia actual
<code>accept-security-baseline</code>	<code>inspector.rs</code>	Se aceptan los controles actuales
<code>accept-tcc-baseline</code>	<code>inspector.rs</code>	Se aceptan los permisos actuales
<code>accept-network-baseline</code>	<code>inspector.rs</code>	Se acepta la red conocida
<code>clean-caches</code>	<code>inspector.rs</code>	Limpieza real, nunca en simulación
<code>terminate-process</code>	<code>inspector.rs</code>	Intento de <code>SIGTERM</code> , con éxito o sin él
<code>reveal-in-finder</code>	<code>inspector.rs</code>	Se revela un archivo
<code>suggest-block-ip</code>	<code>inspector.rs</code>	Se genera la regla <code>pfctl</code>
<code>generate-report</code>	<code>inspector.rs</code>	Se guarda un reporte
<code>ai-explain-latest</code>	<code>inspector.rs</code>	Se consulta la IA opcional
<code>agent-unexpected-stop</code>	<code>resilience.rs</code>	Arranque tras un cierre no limpio
<code>agent-config-changed</code>	<code>resilience.rs</code>	La huella de configuración cambió

agent-clean-shutdown	resilience.rs	Cierre limpio al destruir el servicio
----------------------	---------------	---------------------------------------

audit_log no se recorta. A diferencia de `snapshots` e `incidents`, no hay `trim_table` sobre esta tabla: la auditoría es evidencia y crece indefinidamente. Con el uso esperado son unas pocas filas por sesión, pero se anota en [15 · Riesgos](#).

3.4 persistence_baseline — estado bueno conocido de launchd

Columna	Tipo	Nulo	Descripción
entry_key	TEXT	No	Clave primaria: <code>entry_kind</code> ■ <code>location</code> ■ <code>name</code>
entry_kind	TEXT	No	<code>LaunchAgent</code> , <code>LaunchDaemon</code> , <code>LoginItem</code> , <code>cron</code>
location	TEXT	No	Ruta del plist o descripción del origen
name	TEXT	No	<code>Label</code> del plist o nombre del elemento
command	TEXT	No	Comando efectivo; es el valor que se compara
target_path	TEXT	Sí	Binario destino, si se pudo determinar
first_seen	TEXT	No	<code>CURRENT_TIMESTAMP</code> de la siembra

Por qué la clave no incluye el comando: para que cambiar el comando de un plist existente se lea como *modificación* y no como *entrada eliminada* + *entrada nueva*.

3.5 baseline — estado bueno conocido genérico

Columna	Tipo	Nulo	Descripción
surface	TEXT	No	Clave primaria compuesta: identificador de superficie
entry_key	TEXT	No	Clave primaria compuesta: clave estable del ítem
value	TEXT	No	Valor vigilado; su cambio produce <code>Modified</code>
label	TEXT	No	Nombre legible para la interfaz
detail	TEXT	No	Contexto adicional
first_seen	TEXT	No	<code>CURRENT_TIMESTAMP</code>

Superficies y forma de sus claves y valores:

surface	entry_key	value	Origen
security-control	gatekeeper, sip, filevault, firewall, stealth-mode, remote-login	Activado / Desactivado / Desconocido	security::control_watch_items

tcc-permission	<servicio>::<cliente>	permitido / limitado	tcc::permission_watch_items
network-device	mac:<mac> o ip:<ip>	<ip>\ <es_puerta_de_enlace>	netscan::device_watch_items

4. Diagrama entidad-relación

Diagrama Mermaid #1 — código fuente (no renderizado en PDF)

```
erDiagram
    SNAPSHOTS {
        INTEGER id PK "autoincremental"
        TEXT collected_at "RFC 3339 UTC"
        REAL cpu_usage
        REAL memory_used_gb
        REAL memory_total_gb
        REAL cache_total_mb
        REAL network_rx_mb_delta
        REAL network_tx_mb_delta
        REAL io_read_mb_delta
        REAL io_write_mb_delta
        TEXT dominant_process
        TEXT alerts_json "array JSON de Alert"
        TEXT created_at
    }
    INCIDENTS {
        INTEGER id PK
        TEXT incident_id "inc-<epoch_ms>"
        TEXT fingerprint "deduplicacion"
        TEXT collected_at
        TEXT severity
        TEXT kind
        TEXT title
        TEXT summary
        TEXT payload_json "IncidentSummary completo"
        TEXT created_at
    }
    AUDIT_LOG {
        INTEGER id PK
        TEXT occurred_at
        TEXT action
        TEXT target
        INTEGER success "1 o 0"
        TEXT detail
        TEXT created_at
    }
    PERSISTENCE_BASELINE {
        TEXT entry_key PK "kind + location + name"
        TEXT entry_kind
        TEXT location
        TEXT name
        TEXT command "valor comparado"
        TEXT target_path
        TEXT first_seen
    }
    BASELINE {
        TEXT surface PK
        TEXT entry_key PK
        TEXT value "valor comparado"
        TEXT label
        TEXT detail
        TEXT first_seen
    }
```

Lectura del diagrama. No hay relaciones dibujadas porque **no existen claves foráneas ni referencias entre tablas**. Es una decisión de diseño: cada tabla responde una pregunta distinta y ninguna necesita unirse con otra en tiempo de consulta.

La relación conceptual que sí existe —un incidente nace de una captura— se materializa por marca de tiempo: `incidents.collected_at` coincide con `snapshots.collected_at` de la captura que lo originó, porque ambos se rellenan con el mismo `snapshot.collected_at`. No hay integridad referencial que lo garantice; si `snapshots` recorta esa fila, el incidente sobrevive huérfano. Se documenta como deuda menor en [15 · Riesgos](#).

5. Relación tabla ↔ código

Tabla	Escribe	Lee	Modelo
snapshots	<code>persist_snapshot</code>	<code>load_recent</code> , <code>latest_summary_line</code> , <code>export_history_backup</code>	<code>SystemSnapshot</code> → <code>SnapshotRow</code>
incidents	<code>persist_incident</code> , <code>update_incident_ai</code>	<code>load_recent_incidents</code> , <code>latest_incident</code> , <code>load_incident_by_id</code>	<code>IncidentSummary</code>
audit_log	<code>record_audit</code>	<code>load_recent_audits</code>	<code>AuditRecord</code>
<code>persistence_baseline</code>	<code>replace_persistence_baseline</code>	<code>load_persistence_baseline</code>	<code>PersistenceEntry</code>
baseline	<code>replace_baseline</code>	<code>load_baseline</code>	<code>WatchedItem</code>

Y quién las consume aguas arriba:

Tabla	Sección de la GUI	Comando del CLI
snapshots	Historial · barra de estado	<code>history</code> , <code>history --backup</code>
incidents	Historial · Resumen	<code>incidents</code> , <code>ai explain-latest</code>
audit_log	Historial (pestaña de auditoría)	<code>audit</code>
<code>persistence_baseline</code>	Persistencia (columna Cambio)	<code>persistence</code> , <code>persistence --accept</code>
baseline	Seguridad · Privacidad · Red	<code>security --accept</code> , <code>tcc --accept</code> , <code>network --accept</code>

6. Consultas relevantes

Inserción de captura (`persist_snapshot`)

```
INSERT INTO snapshots (
  collected_at, cpu_usage, memory_used_gb, memory_total_gb, cache_total_mb,
  network_rx_mb_delta, network_tx_mb_delta, io_read_mb_delta, io_write_mb_delta,
  dominant_process, alerts_json
) VALUES (?1, ?2, ?3, ?4, ?5, ?6, ?7, ?8, ?9, ?10, ?11)
```

Deduplicación de incidente (`persist_incident`)

```
SELECT fingerprint FROM incidents ORDER BY id DESC LIMIT 1
```

Si coincide con la huella del incidente entrante, la inserción se omite y el método devuelve `Ok(false)`.

Retención (`trim_table`)

```
DELETE FROM <tabla> WHERE id NOT IN (
  SELECT id FROM <tabla> ORDER BY id DESC LIMIT ?1
)
```


El nombre de tabla se interpola porque SQLite no admite parámetros para identificadores; las dos únicas llamadas pasan literales del propio módulo ("snapshots" e "incidents").

Historial reciente (load_recent)

```
SELECT id, collected_at, cpu_usage, memory_used_gb, memory_total_gb,
       io_write_mb_delta, cache_total_mb, dominant_process, alerts_json
FROM snapshots ORDER BY id DESC LIMIT ?1
```

Reemplazo de baseline (replace_baseline), en transacción:

```
DELETE FROM baseline WHERE surface = ?1;
INSERT OR REPLACE INTO baseline (surface, entry_key, value, label, detail)
VALUES (?1, ?2, ?3, ?4, ?5);
```

7. Transacciones

Solo dos operaciones las usan, y son justo las que deben ser atómicas: replace_persistence_baseline y replace_baseline. Ambas borran e insertan; sin transacción, un fallo a mitad dejaría la baseline vacía, y la captura siguiente reportaría todo como nuevo.

Las inserciones de captura, incidente y auditoría son sentencias únicas: SQLite ya las hace atómicas.

8. Retención y crecimiento

Tabla	Límite	Configurable en
snapshots	1 000 filas	collection.history_limit
incidents	300 filas	collection.incident_limit
audit_log	Sin límite	—
persistence_baseline	Tantas filas como entradas haya	—
baseline	Tantas filas como ítems vigilados	—

INFERENCIA sobre el tamaño: con 1 000 capturas, unas pocas decenas de KB por fila de alerts_json en el peor caso y 300 incidentes con su payload_json, el archivo se mantiene en el orden de unos pocos MB. No se ha medido en una instalación de larga duración: REQUIERE VALIDACIÓN.

9. Datos sensibles almacenados

Dato	Tabla	Sensibilidad	Justificación
Rutas de ejecutables del usuario	snapshots.alerts_json, incidents.payload_json	Media: revelan software instalado y nombre de usuario	Sin la ruta, un hallazgo no es accionable
Nombres de proceso y PID	Ambas	Baja	—
Direcciones IP remotas	incidents.payload_json	Media	Evidencia de la conexión observada
MAC de equipos de la red	baseline	Media: identifican dispositivos	Es la clave de la baseline de red

Identificadores de aplicaciones con permisos TCC	<code>baseline</code>	Media	Necesarios para detectar permisos nuevos
Comandos de plists de persistencia	<code>persistence_ baseline</code>	Media	Es el valor comparado
Claves de API, contraseñas o tokens	—	—	No se almacena ninguno: la clave de IA vive solo en una variable de entorno

Ninguno de estos datos sale del equipo. La única excepción posible es el incidente resumido que viaja al proveedor de IA si el usuario lo activa, y ese payload no incluye permisos TCC ni la captura completa.

10. Respaldo y recuperación

Operación	Cómo
Respaldo del historial	<code>rootcause history --backup → rootcause-history- backup.json</code> junto al SQLite
Respaldo completo	Copiar el archivo <code>.db</code> con la aplicación cerrada
Restauración	Devolver el archivo <code>.db</code> a su ruta
Reinicio total	Borrar el <code>.db</code> ; se recrea vacío en el siguiente arranque
Migración a otro Mac	Copiar toda la carpeta <code>RootCauseInspector</code>

Consecuencia de borrar el `.db`: se pierden historial, incidentes y auditoría, y —lo importante— **las cuatro baselines**. La siguiente captura las siembra de nuevo con el estado actual, así que cualquier cambio anterior a ese momento deja de reportarse. Si el equipo está comprometido, borrar la base es exactamente lo que un atacante querría.

No existe exportación ni importación de baselines por separado: **NO DOCUMENTADO EN EL REPOSITORIO**.

11. Bases de datos ajenas que el producto lee

11.1 `TCC.db` del usuario y del sistema

Aspecto	Detalle
Rutas	<code>~/Library/ Application Support /com.apple.TCC/TCC .db y /Library/ Application Support /com.apple.TCC/TCC .db</code>
Propietario	macOS; formato no documentado por Apple
Apertura	<code>Connection::open_ with_ flags(path, OpenFlags ::SQLITE_OPEN_READ _ONLY)</code>
Escritura	Nunca. No hay una sola sentencia de escritura contra estas bases
Tabla leída	<code>access</code>

Columnas	<code>service</code> , <code>client</code> , <code>auth_value</code> (o <code>allowed</code> en esquemas antiguos), <code>last_modified</code>
Detección de esquema	<code>PRAGMA table_info(access)</code>
Requisito	Acceso total al disco para el binario que ejecuta

Modificar `TCC.db` a mano es una operación que macOS considera hostil y que SIP bloquea; el producto ni lo intenta ni lo sugiere.

11.2 Archivos que actúan como almacén sin ser base de datos

Archivo	Formato	Contenido
<code>rootcause-config.json</code>	JSON	Configuración completa
<code>rootcause-agent-state.json</code>	JSON	<code>AgentStateFile</code> : arranque, latido, cierre limpio, huella de configuración, contador de cierres inesperados, inicio de ventana
<code>rootcause-history-backup.json</code>	JSON	Array de <code>SnapshotRow</code>

12. Verificación manual del contenido

Con `sqlite3` (viene con macOS):

```
sqlite3 ~/Library/Application\ Support/RootCauseInspector/rootcause-history.db ".tables"
sqlite3 ~/Library/Application\ Support/RootCauseInspector/rootcause-history.db \
"SELECT surface, COUNT(*) FROM baseline GROUP BY surface;"
sqlite3 ~/Library/Application\ Support/RootCauseInspector/rootcause-history.db \
"SELECT occurred_at, action, target, success FROM audit_log ORDER BY id DESC LIMIT 10;"
```

Abrir la base mientras la aplicación corre es seguro para lectura; SQLite gestiona el bloqueo.

Siguiente lectura recomendada: [08 · Flujo de datos](#).